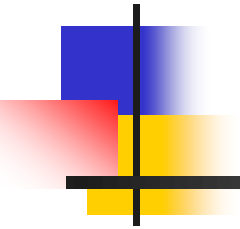


DATA STRUCTURES AND ALGORITHMS

Lecture Notes 3

Pradit Pitaksathienkul

The missing of last exercise

- Python use tab or space to identify a group of statements.

```
def Function(n):
    i = s = 1
    while s < n:
        i = i+1
        s = s+i
        print("*")
```

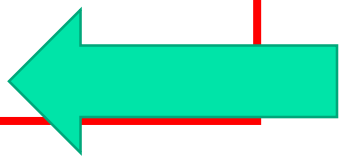
```
def Function(n):
    i = s = 1
    while s < n:
        i = i+1
        s = s+i
    print("*")
```

- It is different !!!

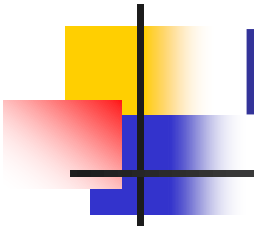
Where is the main program?

- The main program or main function is at the last where out of block of function or class definition.

```
def Function(n):
    i = s = 1
    while s < n:
        i = i+1
        s = s+i
    print("*")

Function(20)
```

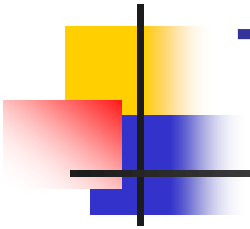


- main program



ROAD MAP

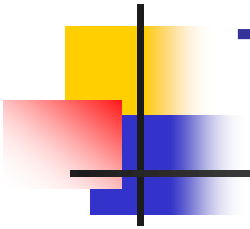
- **Abstract Data Types (ADT)**
  - **The List ADT**
  - **Implementation of Lists**
    - List implementation of lists
    - Linked list implementation of lists



THE LIST ADT

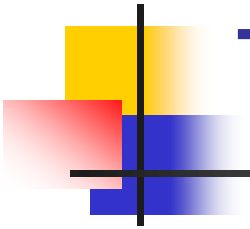
- **Ordered sequence** of data items called elements but **not necessary Sorted.**
- $A_1, A_2, A_3, \ldots, A_N$ is a list of size N
- size of an empty list is 0
- A_{i+1} succeeds A_i
- A_{i-1} preceeds A_i
- position of A_i is i
- first element is A_1 called "head"
- last element is A_N called "tail"

Operations ?



THE LIST ADT

- **Operations**
  - PrintList
  - Find
  - FindKth
  - Insert
  - Delete
  - Next
  - Previous
  - MakeEmpty



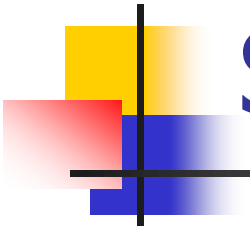
THE LIST ADT

- **Example:**

  the elements of a list are

  34, 12, 52, 16, 15

  - Find (12) → Found
  - Insert (20, 3) → 34, 12, 52, 20, 16, 15
  - Delete (52) → 34, 12, 20, 16, 15
  - FindKth (5) → 15



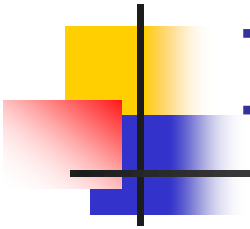
Special case in implementation

- May be special cases we must careful when implementation.
- **Example:**

  the elements of a list are

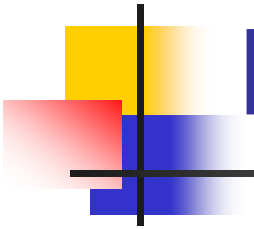
  34, 12, 52, 16, 15

  - Find (22) → ?
  - Insert (20, 10) → 34, 12, 52, 16, 15 ?
  - Delete (60) → 34, 12, 52, 16, 15 ?
  - FindKth (10) → ?



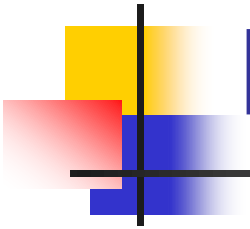
Implementation of Lists

- Many Implementations
  - List (It is one of data type in Python)
  - Linked List



ROAD MAP

- Abstract Data Types (ADT)
  - The List ADT
  - Implementation of Lists
    - **List implementation of lists**
    - Linked list implementation of lists



List implementation of lists

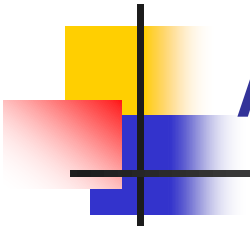
- List is a data type in Python. It is like array in any language.
- List is a group of data which order in List is important.
- It can has 0 to any data.
- When we create List ,we use [ ] .

List implementation of lists

- Example:

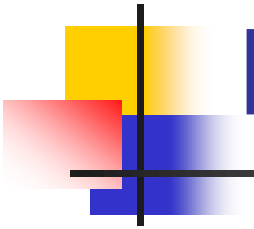
```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]

print('list1 = ',list1)
print('list2 = ',list2)

list2[0] = 37
print('After changing, list2 = ',list2)
```

```
Result:
list1 =  ['apple', 'banana', 'cherry']
list2 =  [1, 5, 7, 9, 3]
After changing, list2 =  [37, 5, 7, 9, 3]
```



Array Implementation of List ADT

- Disadvantages :
  - insertion and deletion is very slow
    - need to move elements of the list
  - redundant memory space
    - it is difficult to estimate the size of array

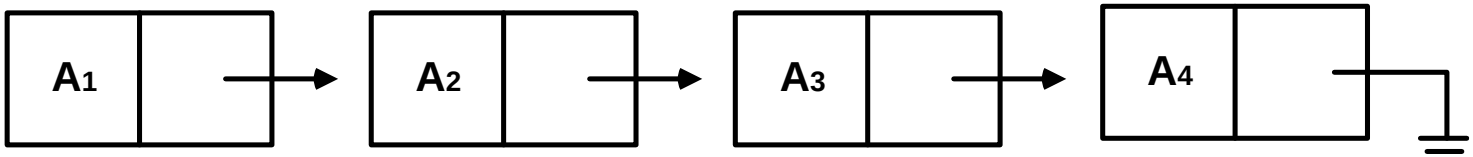


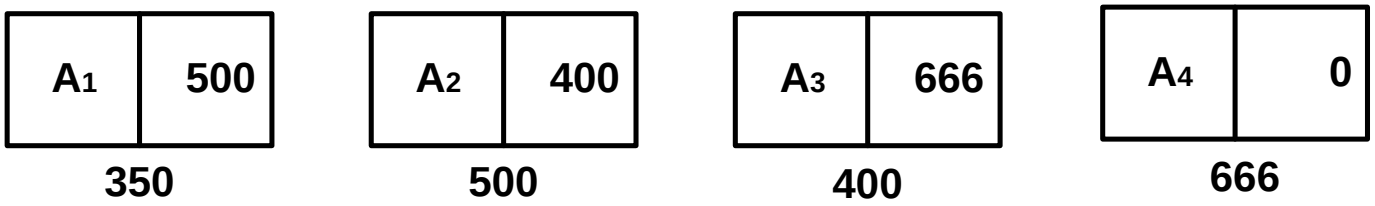
ROAD MAP

- Abstract Data Types (ADT)
  - The List ADT
  - Implementation of Lists
    - List implementation of lists
    - **Linked list implementation of lists**

Linked List Implementation of Lists

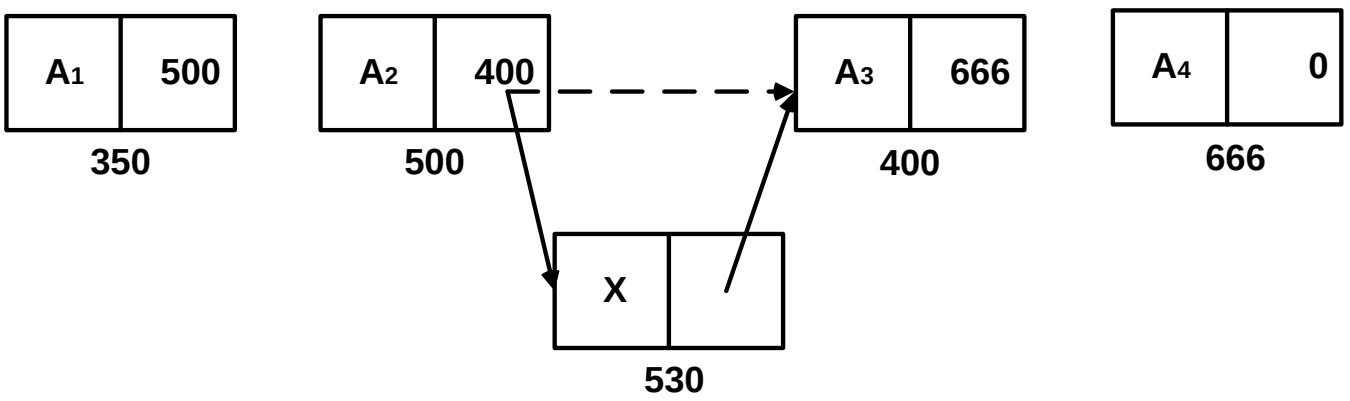
- Series of nodes โหนด
  - not adjacent in memory
  - contain the element and a pointer to a node containing its successor None นัน มีค่า เท่ากับ 0 ในทางคอมพิวเตอร์
- Avoids the linear cost of insertion and deletion !

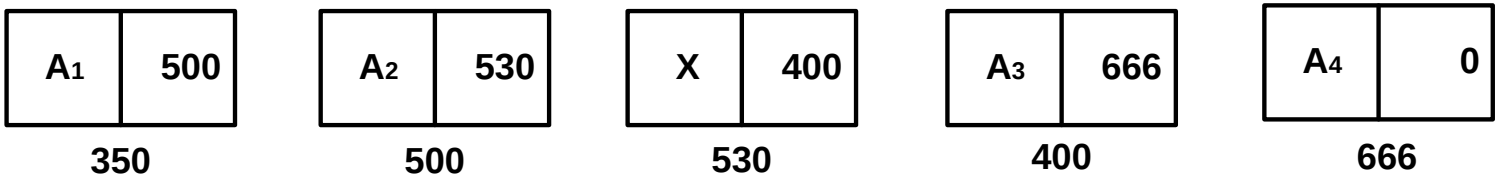




Linked List Implementation of Lists

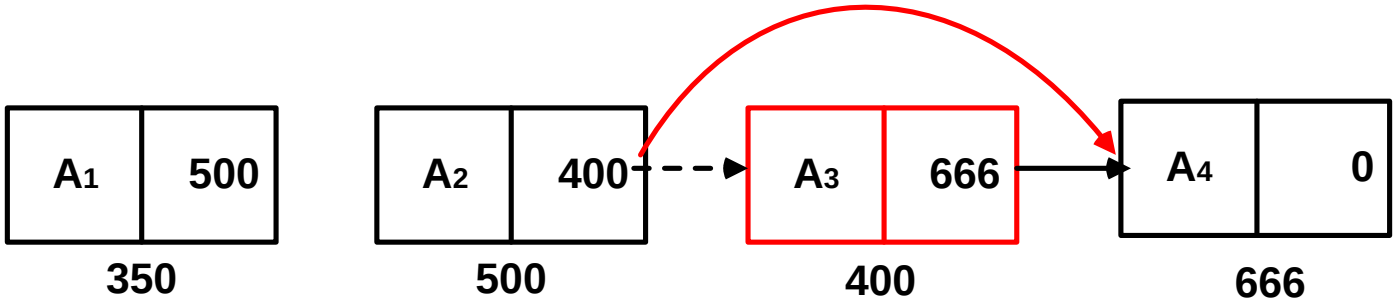
- **Insertion into a linked list**

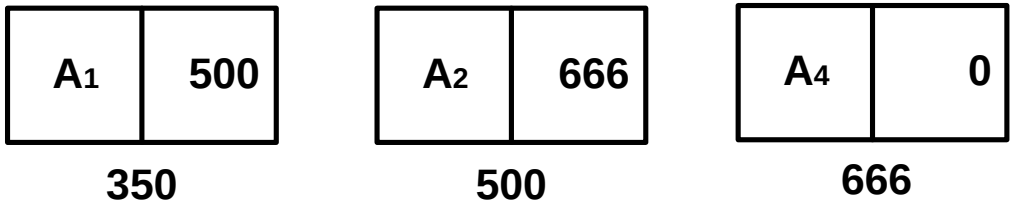


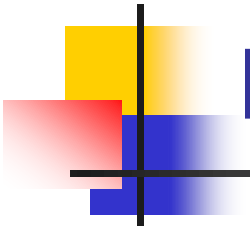


Linked List Implementation of Lists

- **Deletion from a linked list**





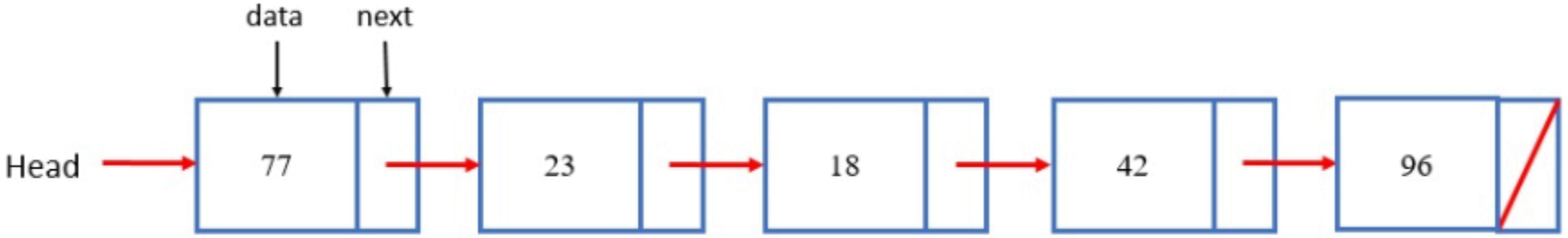


Linked List Implementation of Lists

- Need to know where the first node is
  - the rest of the nodes can be accessed
- No need to move the list for insertion and deletion operations
- No memory waste

Programming Details

Example of LinkedList in Python



- Each node is not adjacent in memory.

Define linked list node

```
class Node:
   def __init__(self,init_data):
      self.data = init_data
      self.next = None
   def get_data(self):
       return self.data
   def get_next(self):
       return self.next
   def set_data(self, new_data):
       self.data = new_date
   def set_next(self, new_next):
       self.next = new_next
```

There are 2 data members in class Node

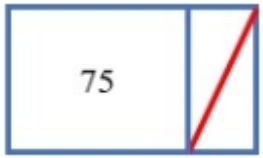
   data + next

There are 5 member functions in class Node

Class for linked lists

```
Example:in main program
temp = Node(75)
print(temp.get_data())
```

```
Result:
75
```




```
Then we define Class LinkedList (Series of Nodes).
class LinkedList:
   def __init__(self):
      self.head = None
   def is_empty(self):
       return self.head == None
```

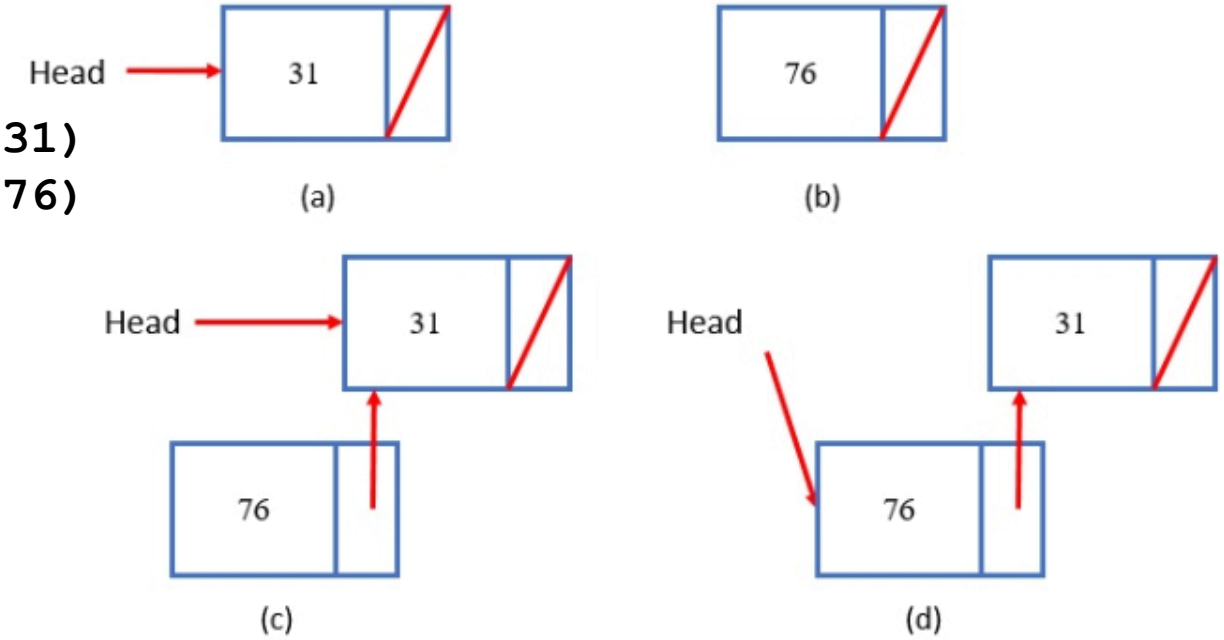
```
Example: in main program
mylist = LinkedList()
```

Add routine

```
def add(self,item):
    temp = Node(item)
    temp.set_next(self.head)
    self.head = temp
```

Example:

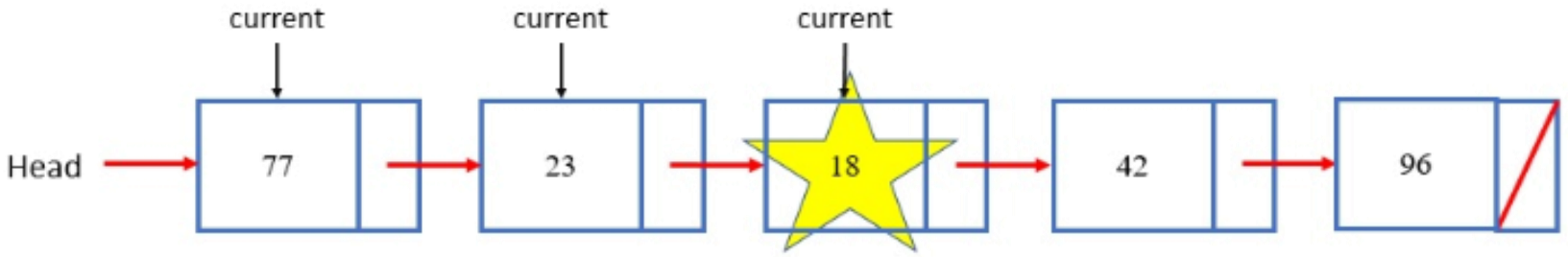
```
mylist.add(31)
mylist.add(76)
```

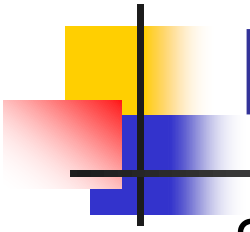


Search routine

```
def search(self,item):
   current = self.head
   found = False
   while current != None and not found:
      if current.get_data() == item:
         found = True
      else:
         current = current.get_next()
   return found
```

Example: If we've a Linked List with data as picture:
mylist.search(18)



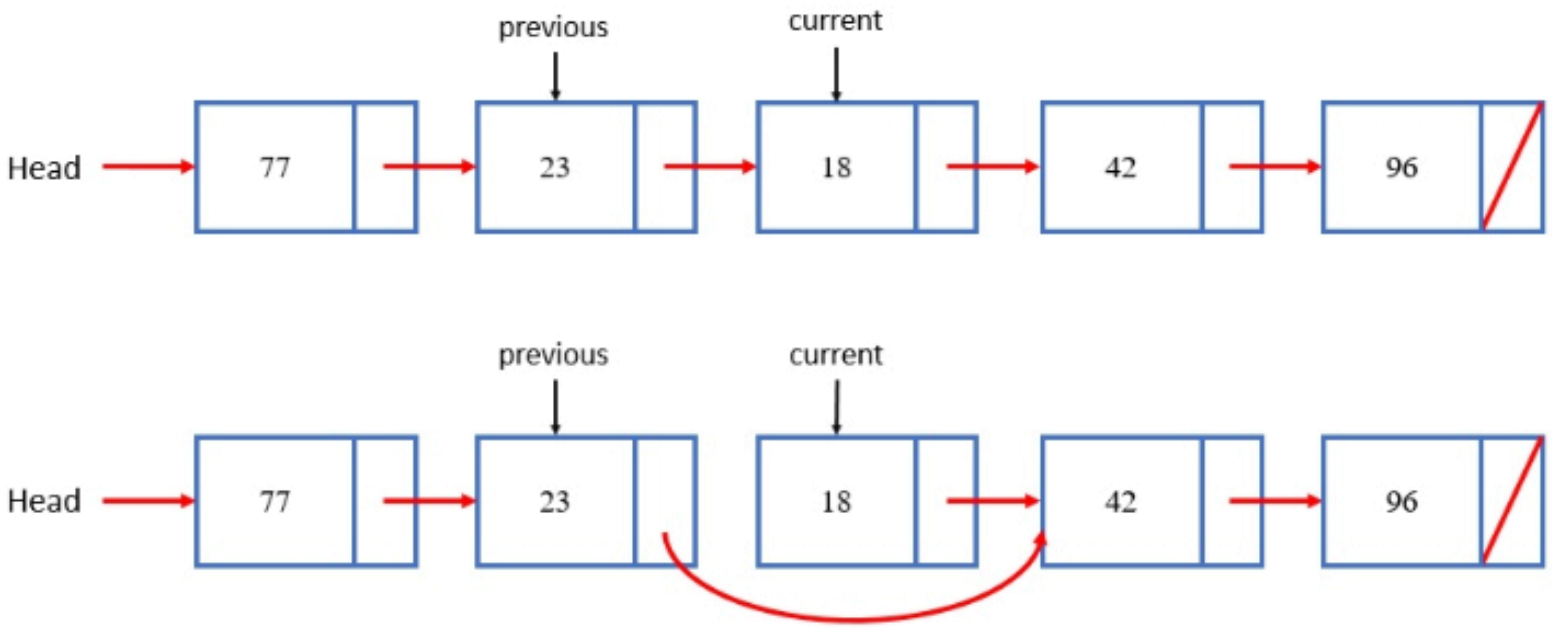


Remove routine

```
def remove(self,item):
    current = self.head
    previous = None
    found = False
    while not found:
        if current.get_data() == item:
            found = True
        else:
            previous = current
            current = current.get_next()
    if not found: return
    if previous == None:
        self.head = current.get_next()
    else:
        previous.set_next(current.get_next())
```

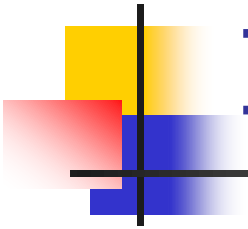
Remove routine

Example:
`mylist.remove(18)`



How to print linked lists

```
def getdata(self):
   current = self.head
   if current != None:
      print(current.get_data(),end=' ')
      current = current.get_next()
   else:
      print("LinkedList is Empty")
      return
   while current != None:
      print(current.get_data())
      current = current.get_next(),end=' '}
```



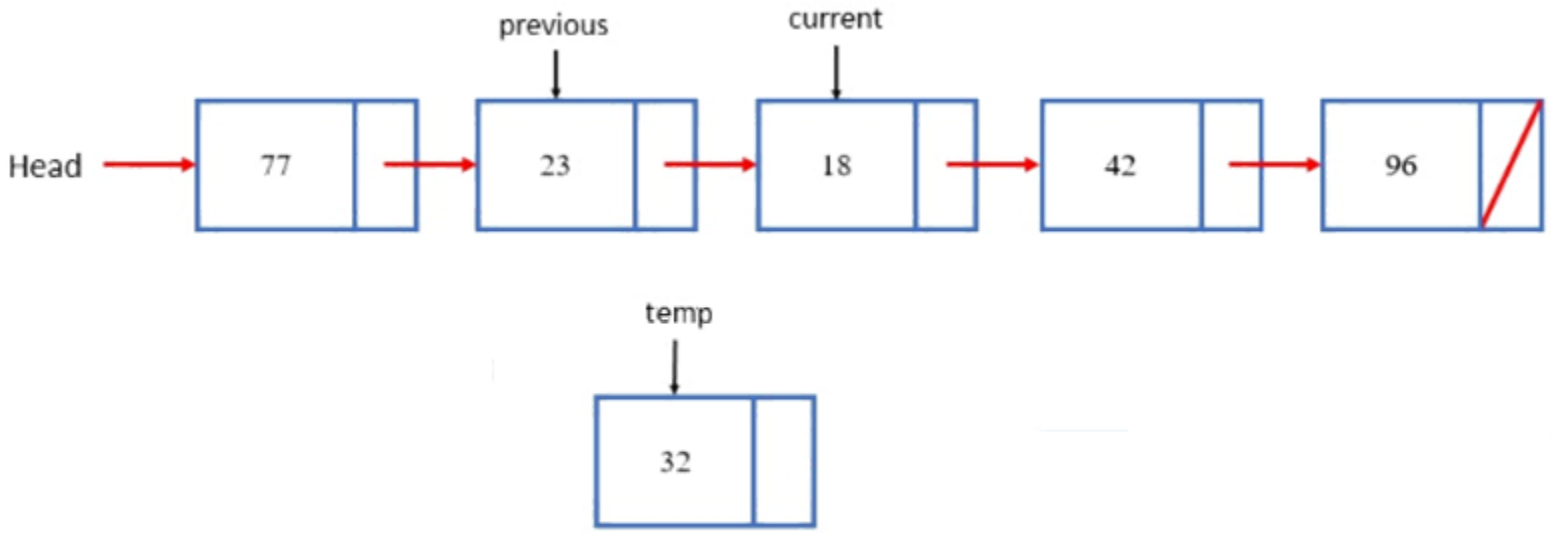
Insert by position

```
def insert(self,item,pos):
   current = self.head
   i = 0
   previous = None
   temp = Node(item)
   for i in range(pos):
       previous = current
       current = current.get_next()
   temp.set_next(current)
   previous.set_next(temp)
```

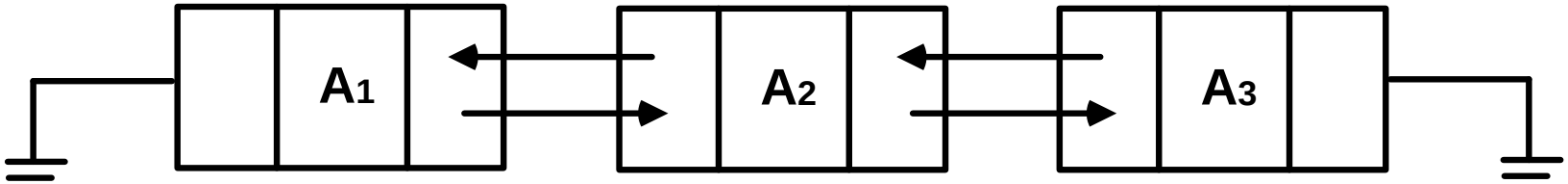
Insert by position

Example:

```
mylist.insert(32,3)
```



Doubly Linked List



- Traversing list backwards
  - not easy with regular lists
- Insertion and deletion more pointer fixing
- Deletion is easier
  - Previous node is easy to find

Circulary Linked List

- Last node points the first